

URL Shortener

Functional/Non-Functional Req. -

Functional Requirements

1. Generate short URLs for a given link.
2. Generate unique short codes for each URL using hashing or other shortening methods.
3. Each generated URL should redirect to the original link.

Non-Functional Requirements

1. Availability: URLs should be available and ready to redirect at all times.
2. Performance: Redirection and generation should be fast (100-300ms).
3. Scalability - The system should support the storage and use of millions of URLs
4. Reliability - Every URL should redirect to the correct original URL without any loss.
5. Security - Use secure redirection methods and prevent misuse using rate limiting.
6. Maintainability - The system should be easy to update and to add new features.

Estimation -

Average New URLs - 100 million/month

Read to Write Ratio - 10:1

Average long URL length - 100 bytes

Short code length - 7 bytes

Metadata (timestamps, location, etc.) - 50 bytes

Total average storage per URL = ~160 bytes

New URLs per day - $100\text{million}/30 = \sim 3.3\text{M}$

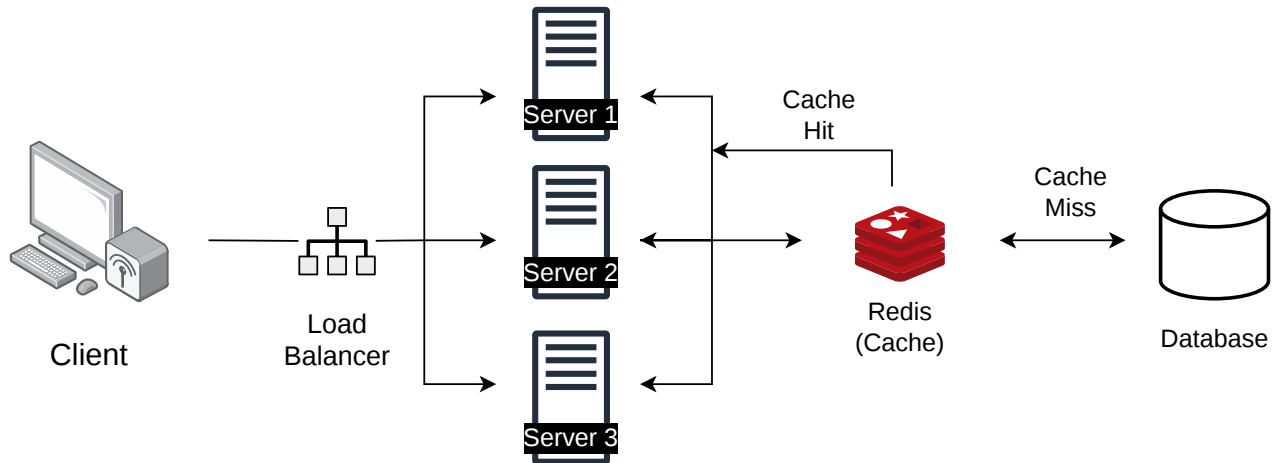
Writes per second = $3.3\text{M}/86400 = \sim 38/\text{sec}$

Reads per second = $38 * 10 = \sim 380/\text{sec}$

Storage per month = $100\text{million} * 160 \text{ bytes} = \sim 16\text{GB}$

Storage per year = $16 * 12 = \sim 192\text{GB}$

Architecture -



Short Code Generation (Alias) -

1. Use Base62 encoding to generate a unique short code.
 - 2 .Convert a unique integer ID into a Base62 string
- Example: 125 → 21 1000 → G8 123456 → W7E

Flow -

Create URL

User → Server → Generate Code → Store Mapping → Return Short URL

Redirect

User → Server → Find Long URL → Increase Click Count → Redirect